PROJET PROGRAMMATION SYSTEME & RESEAU

Exercice 1 : Gestionnaire de mémoire simple

Dans ce laboratoire, vous comprendrez les principes de la gestion de la mémoire en construisant un gestionnaire de mémoire personnalisé pour allouer de la mémoire dynamiquement dans un programme. Plus précisément, vous allez implémenter des fonctions pour allouer et libérer de la mémoire, qui remplacent les fonctions de la bibliothèque C telles que *malloc* et *free*.

Dans cette partie, vous écrirez le code d'un gestionnaire de mémoire, pour allouer et désallouer la mémoire dynamiquement. Votre gestionnaire de mémoire doit gérer 4 Ko de mémoire, en demandant au système d'exploitation une page de 4 Ko via *mmap*. Vous devez prendre en charge les allocations et les désallocations dans des tailles multiples de 8 octets. Le fichier d'en-tête *alloc.h* définit les fonctions que vous devez implémenter. Vous devez compléter votre code dans *alloc.c*. Les fonctions que vous devez implémenter sont décrites ci-dessous.

- La fonction *init_alloc()* doit initialiser le gestionnaire de mémoire, notamment en allouant une page de 4 Ko au système d'exploitation via *mmap* et en initialisant toute autre structure de données nécessaire. Cette fonction sera invoquée par l'utilisateur avant de demander de la mémoire à votre gestionnaire de mémoire. Cette fonction doit renvoyer 0 en cas de succès et un code d'erreur non nul dans le cas contraire.
- La fonction *cleanup()* doit nettoyer l'état de votre gestionnaire et renvoyer la page mappée en mémoire au système d'exploitation. Cette fonction doit renvoyer 0 en cas de succès et un code d'erreur non nul dans le cas contraire.
- La fonction *alloc(int)* prend une taille de tampon entière qui doit être allouée et renvoie un *char** pointeur sur le tampon en cas de succès. Cette fonction renvoie un NULL en cas d'échec (par exemple, la taille demandée n'est pas un multiple de 8 octets, ou l'espace libre est insuffisant). En cas de succès, le pointeur retourné doit pointer vers une adresse mémoire valide dans la page de 4 Ko du gestionnaire de mémoire.
- La fonction *dealloc(char *)* prend un pointeur sur un bloc de mémoire précédemment alloué et libère l'ensemble du bloc.

Il est important de noter que vous ne devez PAS utiliser les fonctions de la bibliothèque C telles que `malloc` pour implémenter la fonction `alloc` ; au lieu de cela, vous devez obtenir une page du système d'exploitation via `mmap` et implémenter vous-même une fonctionnalité telle que `malloc`.
Le gestionnaire de mémoire peut être mis en oeuvre de nombreuses façons. N'hésitez donc pas à le concevoir et à le mettre en oeuvre comme bon vous semble, en respectant les contraintes suivantes.

- Votre gestionnaire de mémoire doit rendre l'intégralité des 4 Ko disponibles pour les allocations à l'utilisateur via la fonction `alloc`. En d'autres termes, vous ne devez pas stocker d'entêtes ou de métadonnées dans la page elle-même, ce qui risquerait de réduire la quantité de mémoire utilisable. Toutes les métadonnées nécessaires au suivi des tailles d'allocation doivent se trouver dans des structures de données définies dans votre code et ne doivent pas être intégrées dans la page de 4 Ko mappée en mémoire elle-même.
- Une fois allouée, une région de mémoire ne doit pas être disponible pour de futures allocations tant qu'elle n'a pas été libérée par l'utilisateur. En d'autres termes, ne réservez pas votre mémoire deux fois, car cela peut détruire l'intégrité des données qui y sont inscrites.
- Une fois qu'un bloc de mémoire de taille $N1$ octets a été désalloué, il doit être disponible pour des allocations de mémoire de taille $N2$ à l'avenir, où $N2 \leq N1$. En outre, si $N2 < N1$, le bloc restant de taille $N1 - N2$ doit être disponible pour des allocations futures. En d'autres termes, votre gestionnaire de mémoire doit avoir la possibilité de diviser un gros morceau libre en morceaux plus petits pour les allocations.
- Si deux blocs de mémoire libres de taille $N1$ et $N2$ sont adjacents, un bloc de mémoire fusionné de taille $N1 + N2$ doit être disponible pour l'allocation. En d'autres termes, vous devez fusionner des blocs de mémoire adjacents et les rendre disponibles pour l'allocation d'un bloc plus grand.
- Après quelques allocations et désallocations, votre page de 4 Ko peut contenir des morceaux alloués et des morceaux libres intercalés les uns avec les autres. Lors de la prochaine demande d'allocation d'un bloc, vous pouvez utiliser n'importe quelle heuristique (par exemple, best fit, first fit, worst fit, etc.) pour allouer un bloc libre, à condition que l'heuristique renvoie correctement un bloc libre s'il existe.

Nous avons fourni un exemple de programme de test *test_alloc.c* pour tester votre implémentation. Ce programme exécute plusieurs tests qui initialisent votre gestionnaire de mémoire et invoquent les fonctions *alloc* et *dealloc* que vous avez implémentées. Notez que nous évaluerons votre code non seulement avec ce programme de test, mais aussi avec d'autres. N'hésitez donc pas à écrire d'autres programmes de test de ce type pour tester votre code.

Exercice 2 : Chat avec Client/serveur parallèle

Le type de serveur donné au cours est un serveur itératif. Une fois que le serveur a accepté une connexion de la part d'un client, les clients suivants sont mis en attente jusqu'à ce que le serveur ait fini de traiter le premier client et revienne sur l'*accept()* pour accepter une nouvelle connexion. Durant tout le traitement d'un client connecté, le serveur travaille sur la socket de service et la socket d'écoute n'est pas utilisée. Il est plus performant de créer un serveur parallèle dont l'architecture logicielle est la suivante :

- un processus père accepte les connexions sur la socket d'écoute et crée une socket de service pour la connexion acceptée ;
- pour chaque connexion acceptée, le processus père crée un processus fils. Ce fils hérite naturellement de la socket d'écoute et de la socket de service ouverte par son père ;
- le processus fils traite la requête du client à travers la socket de service. La socket d'écoute lui est inutile, donc il la ferme ;
- le père est libéré du service du client réalisé par son fils. Il ferme la socket de service et retourne immédiatement accepter de nouvelles connexions sur la socket d'écoute.

Dans ce cas de figure, on a donc autant de fils créés que de connexions acceptées par le père.

Dans ce qui suit, on vous demande de créer une application utilisant ce type de serveur.

Ecrire la partie cliente avec GUI[1]. Cette partie contient une zone de texte mono-ligne dans laquelle on peut entrez les questions suivantes :
1. Comment allez-vous ?
2. Qui est le respo du master CIO/BC/BDA ?
3. Les soutenances auront lieu quand ?
4. Aurevoir

Puis la partie serveur (n'utilisant pas de GUI), répond respectivement aux questions ci-dessus :
1. Je vais bien DIEU fait grâce
2. Dr. ATTA/Dr. KOURAOGO/Dr. EDJA
3. C'est au directeur de la pédagogie d'en décider, aucune idée
4. A bientôt

[1] Tutoriel : Créez une interface avec GTK+ ! (tdct.org) et TP1 - Tutoriel Gtk+2.0/Glade (guillaumeriviere.name) vous aiderons pour la partie cliente.